Summary

AI and the Developer Experience

A journalist who has spent about thirty years interviewing developers (a hobbyist coder since the VIC-20, not a professional developer) argues that AI is changing what developers love and hate about the craft, not just their output: the "constant frustration and bursts of joy" loop is flattening, work is shifting toward reviewing agent output, and assessment rather than writing has become the new bottleneck. He sees coding as the first place at industrial scale where AI can plausibly do valuable labor well, making it a preview for the rest of society: the hard technical skills may be the easiest to automate, while tacit skills — understanding people and mapping their needs onto what can be built — become the valuable ones.

Why developers got into it

- The recurring origin story is near-sorcery: typing a few words and making the machine come alive, often literally an infinite loop printing your name at age twelve.
- Fred Brooks: the programmer, like the poet, works only slightly removed from pure thought-stuff; Danny Hillis joked the work would once have earned a burning at the stake; Erin Sleipner (then at SpaceX): "Coding is the art of telling rocks what to think".
- Kent Beck described the creative act: an image in your mind, then seeing the thing in the world that came directly out of your head.
- An Instagram developer, 48 hours after shipping the first Stories release following a six-month push, saw a stranger using Stories on the train and thought "this is why I do this".
- The darker pleasure: "Constant Frustration and Bursts of Joy" (a chapter title from his book on coders) — a Sisyphus-like loop where every triumph reveals the next problem.
- Max Whitney's bug story: NYU students began logging into each other's accounts; the cause was another university unit consolidating IP addresses into one or two trunks, which collided with session randomness based on login time plus IP address, in uncommented spaghetti code whose author had long left. Figuring it out flipped her from feeling like a fool to "the most powerful person in the universe".

What AI changed for the better

- **Endless conversation:** OpenAI told him Copilot-style coding was an accident — GPT-3 trained on online text turned out to write Python and HTML. Because LLMs are language machines and code has clear syntax, developers now spend their days prompting, cajoling and pleading with "alien intelligences" — ironic for people who joined to avoid endless conversations.
- **Theory of mind for an alien:** Rick Karasick's point — you must build a mental model of a foreign intelligence produced by massive optimization, not a human one.
- **Emotional prompt engineering:** At Hyper Spell, engineer Madam Ebert's cloud instructions included "Pushing code that fails PyTest is unacceptable and embarrassing" — he can't prove it works, but it seemed to help; others write caps-lock pleas like "I will be very, very angry".
- **Raw speed:** Game designer Zach Gage went from one prototype every two or three weeks to dictating a board-game idea into Claude on the school run and having something playable by the time he got home — what took a week now takes five minutes. Tony Chu: the cost of experimentation is much lower, which changes how often you can iterate.
- **Blurring roles:** UX designers ship functioning prototypes instead of Figma handoffs; developers touch front-end; PMs and CEOs show prototypes (not always a good thing).
- **The Christmas inflection:** about a year before the talk, most developers were lukewarm on AI; after Claude Opus and similar models got good around Christmas, people came back with "ninety prototypes," working outside their domain and in unfamiliar languages.
- **Survey signals:** Temporal's survey of 500 engineers — roughly 90% say productivity is up, half say prototyping is orders of magnitude faster; GitLab's AI adoptability survey of 1.5 million developers — nearly 80% report higher productivity; Nielsen Norman's UX developers average 4.2 on a 0–5 speed scale.
- **Where tokens are free:** Anthropic's internal paper reports commits per engineer up 5.1x over the last year — far larger than sales' gains.
- **Size matters:** small greenfield startups report things like "twenty X" (per a former Uber engineer now at a startup); Google, with billions of lines and uptime obligations, reports about ten percent delivery improvement — a massive win given its safeguards.
- **Learning tool:** an Amazon developer re-implemented a microservice in Rust over a weekend without knowing Rust, after the original took three months.
- **Soft skills rise:** work abstracts upward toward strategy — at Dropbox, strategy went from about 20% of developers' time to more like 80–90%. Northeastern's analysis of five years of job postings shows hiring shifting from narrow technical skill sets to problem solving and communication.
- **Quality, not speed:** a startup still takes three weeks per feature, but now has a working first iteration on day one and iterates with customer feedback throughout, so it ends confident it built the right thing.

What it costs

- **Craft and joy lost:** a senior developer (unnamed at his company's request, since they push AI hard) asks why he should offload "the exciting, interesting part" — solving the little problems — and says the joy is gone. Less frustration also means less of the dynamic excitement.
- **Management and review load:** heavy agent use turns developers into managers/editors of an army of bots; one Anthropic paper quote compares it to a daycare of children all demanding juice boxes at the same urgency. Assessing agent output is harder than reviewing a colleague's code, where you know their style.
- **Less writing, more judging:** DevRel's survey of about 700 developers found 79% spend less than half their time writing code from scratch.
- **The new bottleneck:** a Harvard study analyzing about 300 million work events (commits, PRs, Jira issues and epics) across roughly 700,000 people at 700 firms found more commits and PRs but epics and issues closing only about one or two percent faster — energy absorbed by managing and judging output. DevRel CEO Darren Chika: the saved code-generation time was swallowed by assessment overhead.
- **Confident wrongness:** AI output reads smooth even when wrong, so errors don't follow the predictable human patterns reviewers look for; reading AI code becomes high-cognitive-load debugging rather than a logic check.
- **Decision fatigue and the vigilance effect:** rapid-fire decisions burn people out, and staying alert to a system that is right most of the time but occasionally wrong is very hard.
- **De-skilling:** seniors worry they are losing the sense built by doing things by hand. One developer compared it to Tesla self-driving: as the car got better he drove less, and the human gets handed the wheel precisely in a crisis — the German word for having knowledge at your fingertips came up.

The counterargument and the history

- Tim Bozarth (Microsoft Engineering Horizons, a woodworker taught by his father): woodworkers using CNC machines are still doing good woodworking, with the ideas and design coming from the maker — he sees AI the same way.
- Val Orsov of vCreate Crafts: coding was how he got to make things; no longer bottlenecked by typing speed, he prototypes and iterates far faster and gets the thrill sooner.
- The 1960s abstraction jump: Grace Hopper pushed for an English-like language plus a compiler, producing FLOW-MATIC and then COBOL — which succeeded and still runs banks and insurers, despite mockery for verbosity.
- Objections then sound familiar: compilers won't be as efficient as humans, and you won't really understand the system unless you are close to the metal with registers and memory.
- Dijkstra on BASIC: students with prior exposure are "mentally mutilated beyond hope of regeneration" (and he said worse about COBOL) — yet many who started on BASIC in the 1980s became phenomenal bare-metal programmers.

- What is genuinely new: code pushed to production that nobody has read — different from the last twenty years of trusting unread dependencies, as in the XKCD cartoon about the random person in Nebraska maintaining a package since 2003.

Jobs, juniors and the future

- Big tech is capital-centric and will replace labor with capital where it can; Microsoft offered buyouts to a large chunk of staff a week before the talk to fund data-center investment.
- Stanford Digital Economy Lab finds no widespread economy-wide displacement so far, with one exception: junior roles in AI-exposed areas, coding among them, are down about 19% versus what would normally be expected this year.
- Why: at larger firms AI now does the scut work juniors used to do, faster and cheaper, and onboarding a junior has been estimated at $17,000 on top of salary.
- The worry is choking off the next generation: experienced developers crawled over broken glass line by line to build code sense, which is what lets them judge AI output.
- Duke's Carmen Tomassi tells students you can fake your way through a CS degree with AI, but you have to want to do the hard work yourself — sending a robot to the gym lifts the weight for you and you gain no muscle.
- The airline analogy: 1990s autopilot softened pilot skills, so airlines mandated a set amount of hands-on flying each week; some developers suggest something similar for coding.
- Two competing forecasts, with no consensus among the dozens of analysts and economists he has spoken to: jobs near AI shrink as capital gets cheaper, or the Jevons effect applies — cheaper software means far more of it, as with lighting moving from scarce to everywhere, including car undersides and children's sneakers. Delivery would likely shift toward many smaller companies or embedded roles, possibly with lower pay.

Closing observation

- Temporal's survey of 554 developers found a "shaky calm": respondents are more likely to say they are less stressed than a year ago, and even developers annoyed by the shift are engaged in finding a way through.
- His thesis: the technical, "hard" skills may be the easiest to automate, while tacit knowledge — dealing with people, understanding their needs, mapping them onto what is technically possible — is the trickier and more valuable part.
- Coding is the first and only place so far where AI plausibly does industrial-scale valuable labor well; what this room works out should illuminate the path for medicine, hospitality and the rest of society.
- He invited anyone who wants to share their experience to find him after the talk.

Next steps

- Anyone willing to share what they are going through with AI and development should track down the speaker after the talk.